Task 1: Sentiment Analysis on Product Reviews

Full Explanation — Python Notebook Walkthrough

Overview

The goal of this task is to build a binary text classifier that reads product or movie reviews and decides whether the sentiment is Positive or Negative. The project uses the IMDb Reviews dataset and follows the standard NLP pipeline: clean the text, convert it to numbers, train a model, and evaluate its results.

Pipeline at a Glance

Step	What happens
1. Load data	Read IMDB Dataset.csv into a Pandas DataFrame
2. Clean text	Remove HTML tags, punctuation, stopwords; lowercase everything
3. Encode labels	Map 'positive' → 1 and 'negative' → 0
4. Split data	80% for training, 20% for testing
5. TF-IDF	Convert cleaned text to a numeric matrix (top 10,000 words)
6. Train model	Fit Logistic Regression on the training vectors
7. Evaluate	Measure accuracy, precision, recall, F1-score on test set
8. Compare	Repeat with Naive Bayes and compare accuracy
9. Visualize	Bar charts of the most frequent positive and negative words

Section 1 — Imports & Setup

We import the libraries we need:

- pandas — load and manipulate the CSV dataset
- re — regular expressions for text cleaning
- nltk — provides the English stopword list
- sklearn — TF-IDF vectorizer, classifiers, and evaluation metrics

We download the stopwords list once:

```
nltk.download('stopwords')
STOPWORDS = set(stopwords.words('english'))
```

Storing stopwords in a set makes lookup O(1) instead of O(n) — much faster on large datasets.

Section 2 — Load Dataset

The IMDb dataset (available on Kaggle) is a CSV file with two columns: review (the text) and sentiment ('positive' or 'negative'). We read it with:

```
df = pd.read_csv('IMDB Dataset.csv')
```

The dataset has 50,000 rows — 25,000 positive and 25,000 negative — making it perfectly balanced, so we do not need to worry about class imbalance.

Section 3 — Text Preprocessing

Raw review text contains noise that hurts model accuracy. The clean_text function applies four steps in sequence:

Step A — Remove HTML tags

```
text = re.sub(r'<.*?>', '', text)
```

IMDb reviews often contain HTML like
 or <b>. The regex <.*?> matches any HTML tag and replaces it with nothing.

Step B — Keep only letters

```
text = re.sub(r'[^a-zA-Z]', ' ', text)
```

Numbers and punctuation do not carry sentiment information, so we replace everything that is not a letter with a space.

Step C — Lowercase and split

```
words = text.lower().split()
```

Lowercasing ensures 'Good' and 'good' are treated as the same word.

Step D — Remove stopwords

```
words = [w for w in words if w not in STOPWORDS]
```

Stopwords like 'the', 'is', 'a', 'and' appear in every review regardless of sentiment and add no value to classification.

The cleaned text is stored in a new column called clean_review.

Section 4 — Encode Labels

Machine learning models work with numbers, not strings. We convert the sentiment column:

```
df['label'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

Now every row has a numeric label: 1 = positive review, 0 = negative review.

Section 5 — Train / Test Split

We split the data so the model learns from one part and is tested on an unseen part:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

- test_size=0.2 means 20% (10,000 reviews) are held out for testing
- random_state=42 makes the split reproducible — the same split every run

Section 6 — TF-IDF Vectorization

Models cannot process raw text — we must convert words into numbers. TF-IDF (Term Frequency–Inverse Document Frequency) does this:

- TF (Term Frequency): how often a word appears in a single review
- IDF (Inverse Document Frequency): penalises words that appear in almost every review (e.g. 'film') because they carry less meaning
- The final score = TF × IDF — rare but locally frequent words get a high score

```
tfidf = TfidfVectorizer(max_features=10000)
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec  = tfidf.transform(X_test)
```

fit_transform is called on the training set to learn the vocabulary. transform (without fit) is called on the test set so that no information from the test data leaks into the model.

Section 7 — Logistic Regression

Logistic Regression is a simple but powerful linear classifier. For each review it calculates a score, applies the sigmoid function to get a probability between 0 and 1, and predicts positive if the probability is > 0.5.

```
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_vec, y_train)
```

max_iter=1000 gives the optimiser enough iterations to converge on a 10,000-feature space. After training we evaluate:

```
print(classification_report(y_test, y_pred_lr))
```

The report shows Precision (of all positives predicted, how many were correct), Recall (of all actual positives, how many did we catch), and F1-score (the harmonic mean of both). On IMDb, Logistic Regression typically reaches ~89% accuracy.

Section 8 — Naive Bayes Comparison (Bonus)

Naive Bayes assumes all words are independent of each other (the 'naive' assumption). Despite this simplification, it works surprisingly well for text classification:

```
nb = MultinomialNB()
nb.fit(X_train_vec, y_train)
```

MultinomialNB is designed for count-based or TF-IDF features. It trains much faster than Logistic Regression but is usually slightly less accurate. On IMDb, it typically scores around ~85% — about 4 points lower than LR.

Section 9 — Word Frequency Visualization (Bonus)

We visualise the most common words in positive and negative reviews using a horizontal bar chart. The top_words function:

- Filters the DataFrame to rows with the chosen label
- Joins all cleaned reviews into one large string
- Uses Counter.most_common(15) to get the 15 highest-frequency words

The chart gives an intuitive sanity check — positive words like 'great', 'love', 'best' should dominate the positive chart, while words like 'bad', 'worst', 'boring' dominate the negative one.

Section 10 — Predict a Custom Review

The predict_review function lets you test the model on any sentence you write:

```
def predict_review(text):
    cleaned = clean_text(text)
    vec = tfidf.transform([cleaned])
    pred = lr.predict(vec)[0]
    return 'Positive' if pred == 1 else 'Negative'
```

It applies the same cleaning pipeline, vectorises with the already-fitted TF-IDF (no refit), and returns the label from the trained model.

Key Concepts Summary

Concept	Plain-English meaning
Stopwords	Very common words (the, is, a) removed to reduce noise
TF-IDF	Numeric score for each word that rewards rare but locally frequent terms
fit vs transform	fit learns parameters; transform applies them — only fit on training data
Logistic Regression	Linear model that outputs a probability via the sigmoid function
Naive Bayes	Probabilistic model assuming word independence — fast, slightly less accurate
Accuracy	Percentage of correct predictions over total predictions
F1-score	Harmonic mean of precision and recall — useful for balanced datasets